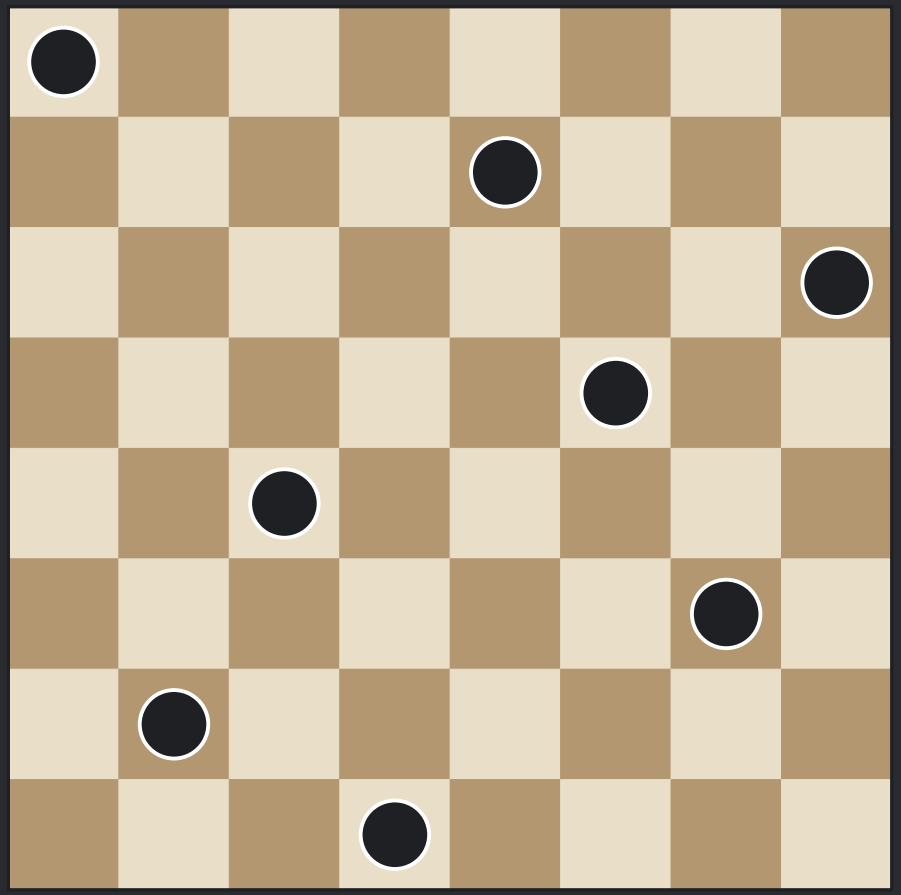
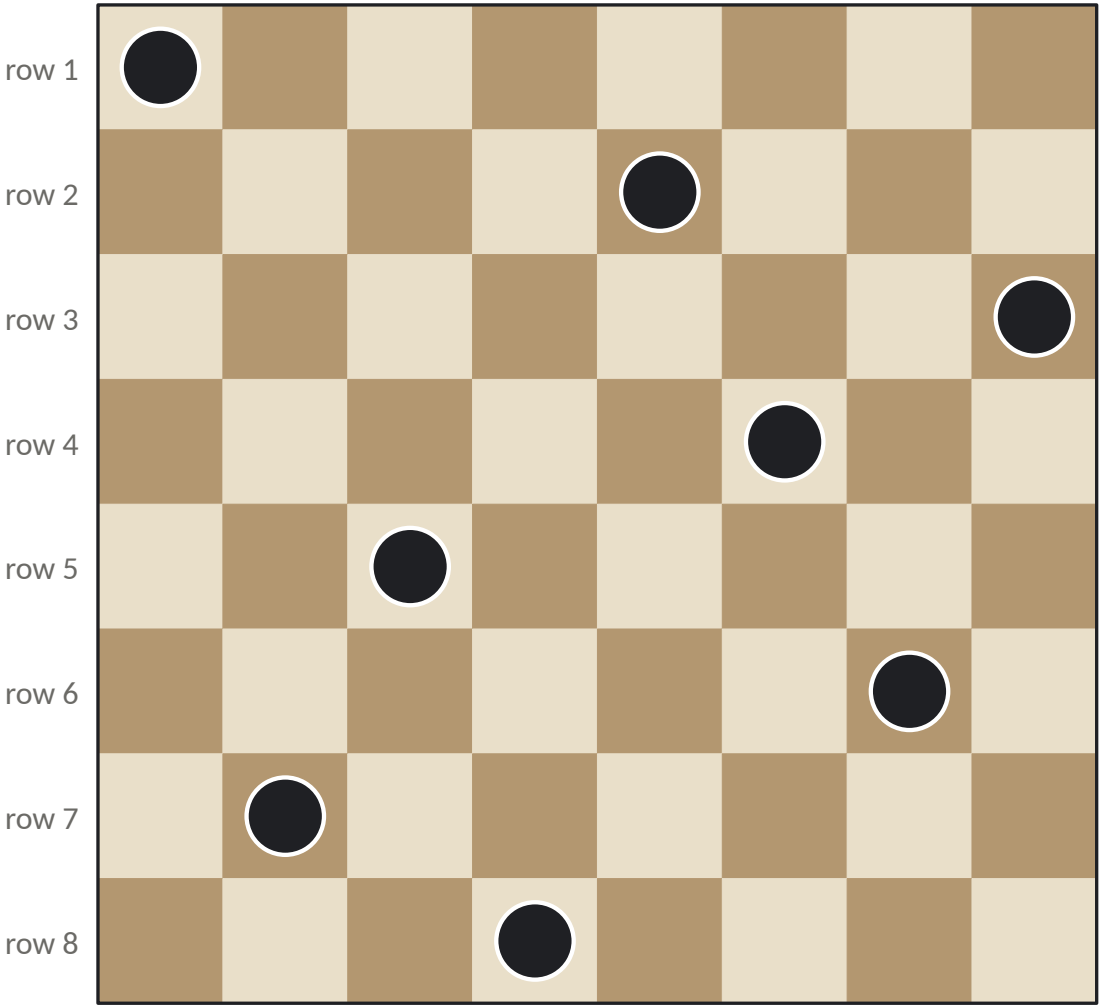
N Queens

One constraint satisfaction problem, solved twice:
clp(fd) in Prolog and python-constraint in Python



Place N queens so that none attacks another



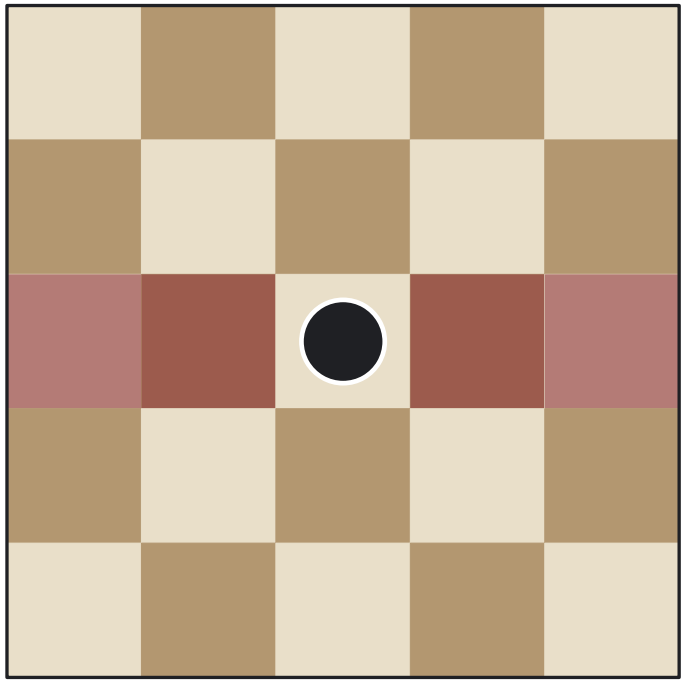
A queen attacks along its row, its column and both diagonals, any distance.

The board on the left is one of the 92 ways to place 8 queens.

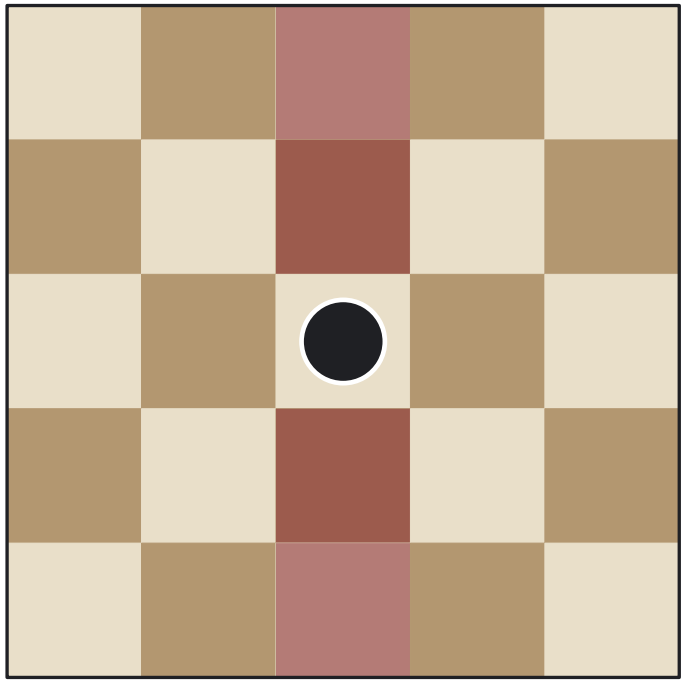
There is at least one solution for every N except 2 and 3.

Posed for N = 8 in 1848; a standard benchmark for search and constraint solvers ever since.

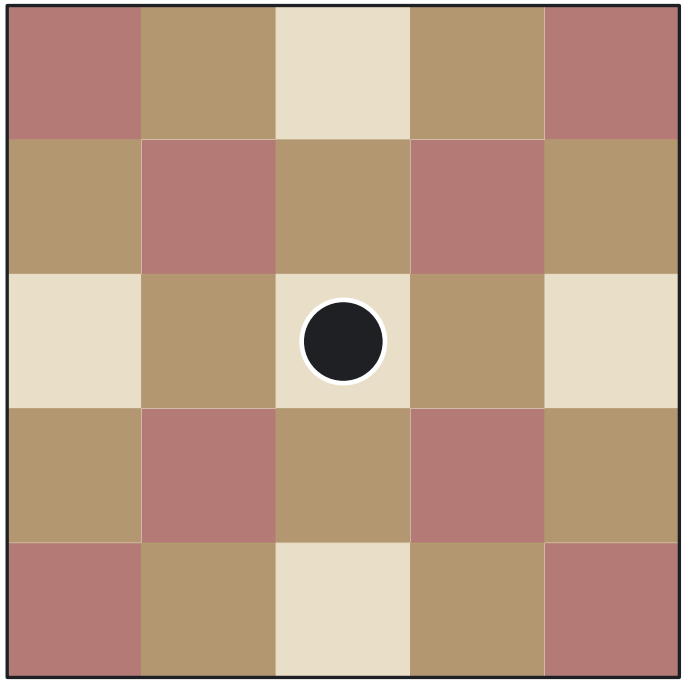
Two queens clash on a row, a column or a diagonal



Same row



Same column



Same diagonal

Diagonal rule in numbers: queens in rows i and j clash when $|Qi - Qj| = |i - j|$.

The choice of variables decides the size of the search

Representation for N = 8	Count	Candidate boards
Any 8 of the 64 squares	`C(64, 8)`	4,426,165,368
One variable per row: its column	`8^8`	16,777,216
One per row, all columns different	`8!`	40,320
Actual solutions		**92**

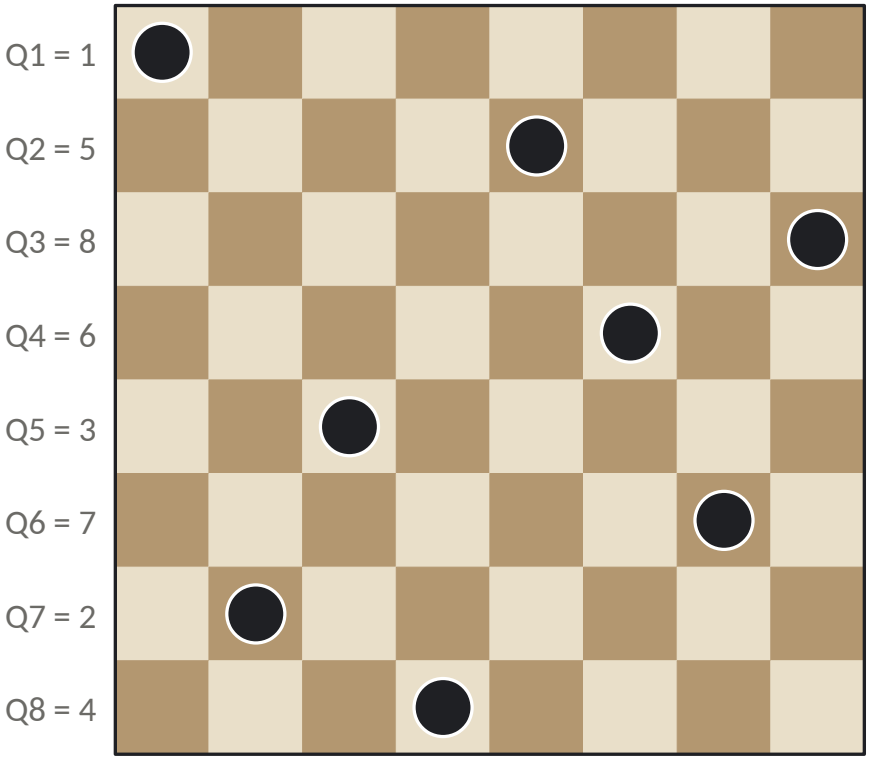
One variable per row makes row conflicts impossible by construction. What is left to state are columns and diagonals.

The CSP, written down

Variables
Q1, Q2, …, QN — Qi is the column of the queen in row i

Domains
{1, 2, …, N} for every Qi

Constraints, for every pair of rows i < j
`Qi ≠ Qj`
`|Qi − Qj| ≠ j − i`



`[1, 5, 8, 6, 3, 7, 2, 4]`

Diagonals become numbers

2	3	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	12
6	7	8	9	10	11	12	13
7	8	9	10	11	12	13	14
8	9	10	11	12	13	14	15
9	10	11	12	13	14	15	16

`row + column`

0	1	2	3	4	5	6	7
-1	0	1	2	3	4	5	6
-2	-1	0	1	2	3	4	5
-3	-2	-1	0	1	2	3	4
-4	-3	-2	-1	0	1	2	3
-5	-4	-3	-2	-1	0	1	2
-6	-5	-4	-3	-2	-1	0	1
-7	-6	-5	-4	-3	-2	-1	0

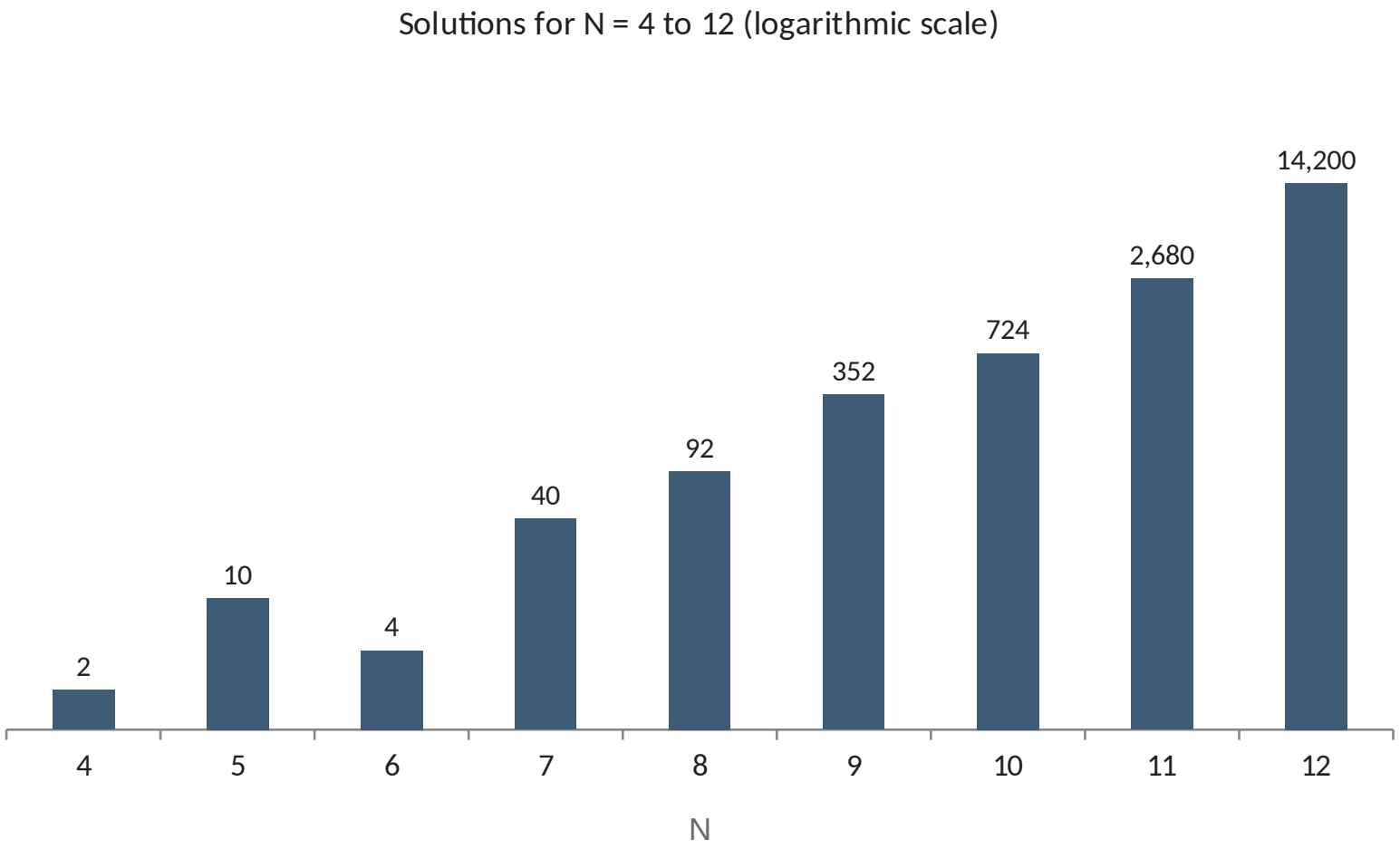
`column − row`

Along each / diagonal the sum is constant; along each \ diagonal the difference is constant.

So instead of N(N−1)/2 pairwise tests, three global constraints are enough:

```
all different Qi
all different Qi + i
all different Qi − i
```

How many solutions are there?

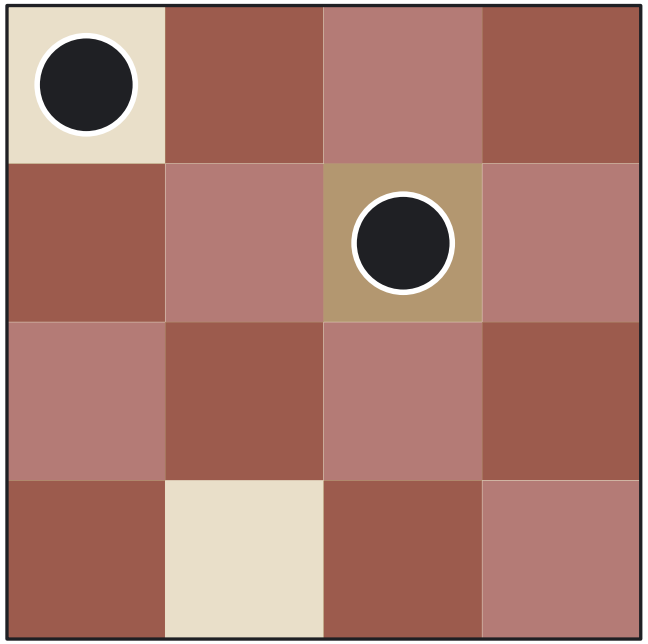


The count is irregular: N = 6 has fewer solutions than N = 5.

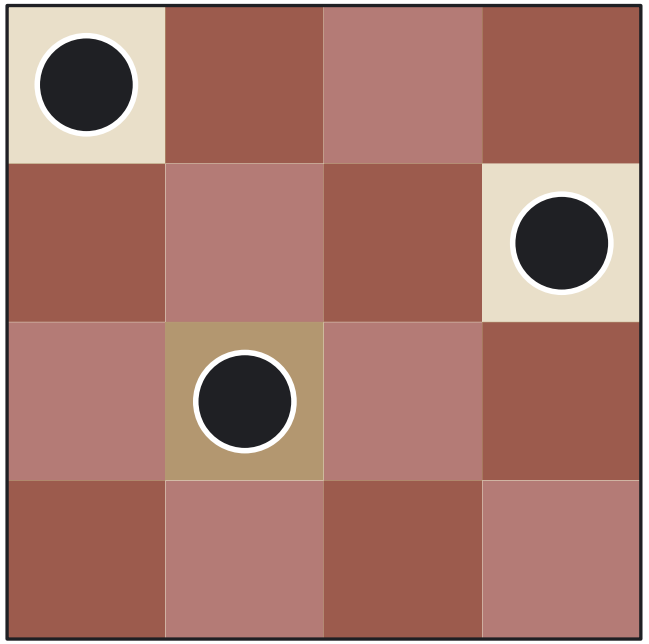
Of the 92 for N = 8, only 12 are truly different; the rest are rotations and reflections.

Counted with both programs; they agree.

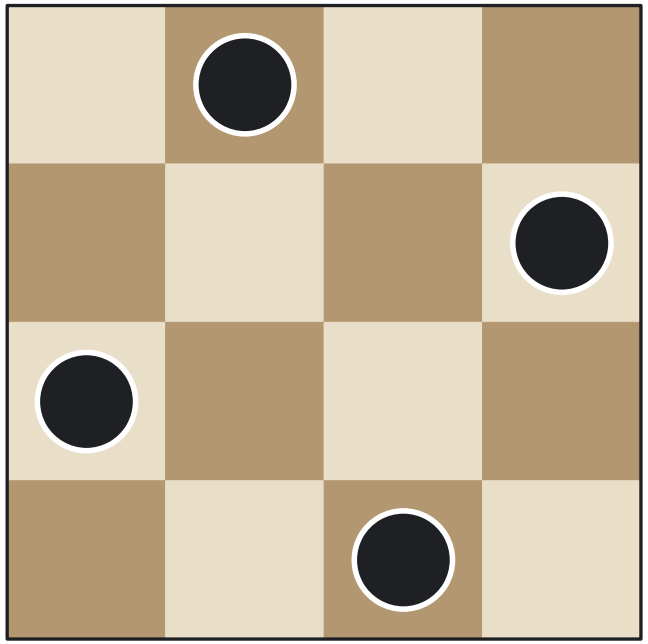
Plain backtracking on N = 4



Q1 = 1, Q2 = 3: every square of row 3 is attacked. Back up.



Q2 = 4, Q3 = 2: now row 4 is fully attacked. Back up to row 1.

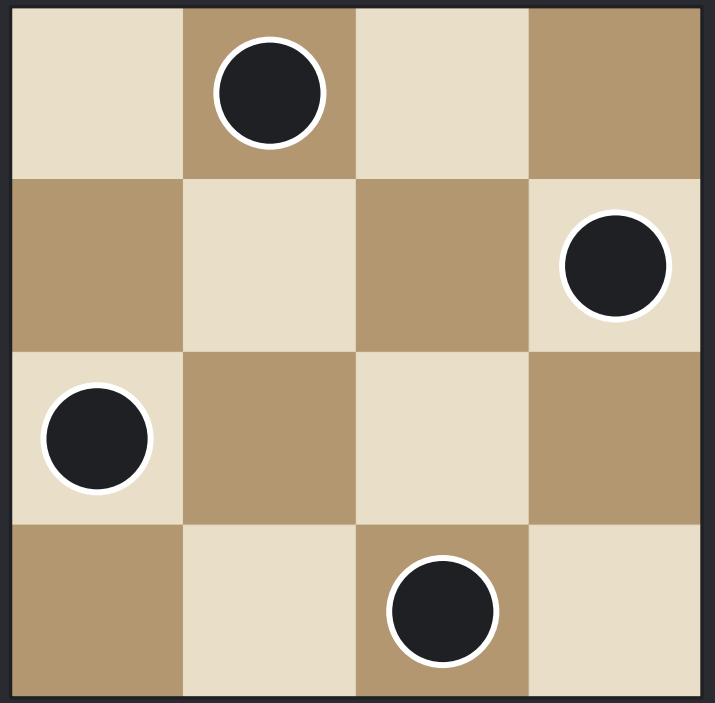


Q1 = 2 leads straight to a solution: [2, 4, 1, 3].

Shaded: squares attacked by the queens already placed.

Prolog with clp(fd)

Constraint logic programming over finite domains: library(clpfd), included with SWI-Prolog



What clp(fd) adds to Prolog

Goal	Meaning
`Xs ins 1..N`	every variable in Xs takes a value from 1 to N
`X #= Y + 2`	arithmetic constraint; works in both directions
`X #\= Y`	X and Y must differ
`all_different(Xs)`	all values in Xs must differ (one global constraint)
`labeling(Opts, Xs)`	search: give values to Xs, backtracking when needed

```
:- use_module(library(clpfd)).
```

Constraints work before the values are known

```
?- X #= 3 + 4.
X = 7.

?- 7 #= 3 + Y.
Y = 4.

?- X in 1..10, X #> 7.
X in 8..10.
```

is/2 can only compute X from known values. #= can solve for any side.

When there is no single answer, Prolog answers with what remains: a smaller domain. That is propagation, the same idea as forward checking.

The model: constraints only, no search

```
constrain(N, Cols) :-
    length(Cols, N),                % one variable per row
    Cols ins 1..N,                  % each one is a column
    all_different(Cols),            % different columns
    numlist(1, N, Rows),
    maplist(sum_with_index, Cols, Rows, Sums),
    maplist(diff_with_index, Cols, Rows, Diffs),
    all_different(Sums),            % different / diagonals
    all_different(Diffs).           % different \ diagonals

sum_with_index(Q, I, S)  :- S #= Q + I.
diff_with_index(Q, I, D) :- D #= Q - I.
```

`length/2` creates one fresh variable per row.

`maplist/4` builds the lists of Qi + i and Qi − i, as constrained variables.

Three all_different goals state the whole problem.

Then search, with one call

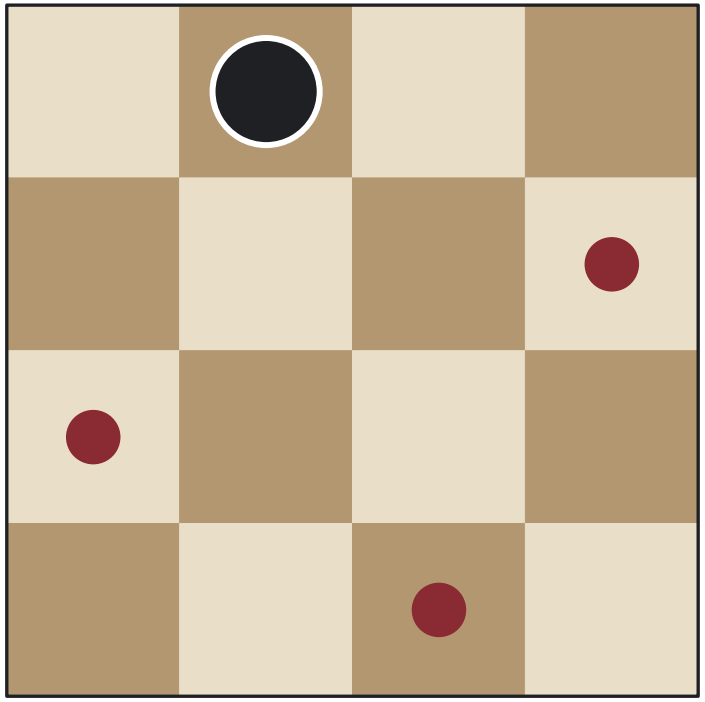
```
queens(N, Cols, Options) :-
    constrain(N, Cols),
    labeling(Options, Cols).
```

labeling/2 does the backtracking. Asking for more answers with ; enumerates every solution.

```
?- queens(8, Cols, [ff]).
Cols = [1, 5, 8, 6, 3, 7, 2, 4] ;
Cols = [1, 6, 8, 3, 7, 4, 2, 5] ;
...
```

Separating the model from the search makes it easy to change one without touching the other.

Propagation alone can solve the board

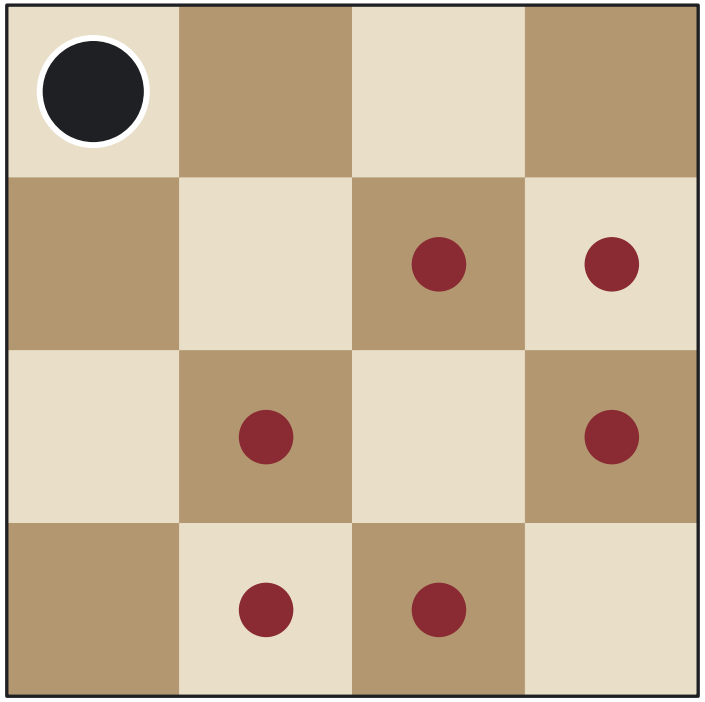


```
?- constrain(4, Cols), Cols = [Q1|_], Q1 = 2,
   maplist(fd_dom, Cols, Domains).
Domains = [2..2, 4..4, 1..1, 3..3].
```

We only fixed the first queen. Every other domain shrank to a single value, without calling labeling.

Dots: the values still left in each row.

…but not always



```
?- constrain(4, Cols), Cols = [Q1|_], Q1 = 1,
   maplist(fd_dom, Cols, Domains).
Domains = [1..1, 3..4, 2\/4, 2..3].

?- ..., labeling([leftmost], Cols).
false.
```

With Q1 = 1 no domain is empty, yet there is no solution. all_different only removes values it can prove impossible; search has to do the rest.

Forget labeling, and Prolog answers with constraints

```
?- constrain(4, Cols).
Cols = [_A, _B, _C, _D],
_A in 1..4,
all_different([_A, _B, _C, _D]),
_E+1#=_A,
_A+1#=_F,
...
all_different([_F, _H, _J, _L]),
...
all_different([_E, _G, _I, _K]),
...
```

No error and no values: the answer is the set of pending constraints.

This is the most common mistake with clp(fd). If the output shows `in` and #=, you forgot to call labeling.

Labeling options are the heuristics from class

Option	Which variable next	In the CSP slides
`leftmost`	the first unassigned one, in order	no heuristic
`ff`	the one with the smallest domain	MRV (first fail)
`ffc`	smallest domain, ties broken by most constraints	MRV + degree
`min / max`	the one with the smallest / largest lower bound	—

Values are tried in increasing order by default (up); down reverses it.

First fail is the difference between instant and stuck

N	leftmost	ff
8	0.006 s	0.005 s
16	0.61 s	0.014 s
20	13.0 s	0.024 s
22	> 20 s	0.008 s
50	> 20 s	0.042 s
100	> 20 s	0.46 s

Time to the first solution, SWI-Prolog 9, limit 20 s.

100

queens placed in half a second with ff; leftmost gives up before 22

Search times are not smooth

N = 20	N = 21	N = 150	N = 175
13.0 s	0.59 s	> 20 s	0.62 s
`leftmost`	`leftmost`	`ff`	`ff`

A larger board can be much faster. The time depends on how early in the search order the first solution sits, and one bad early choice can cost an exponential detour.

Counting every solution

```
count_solutions(N, Count) :-
    aggregate_all(count, queens(N, _, [leftmost]), Count).
```

N	solutions	time
8	92	0.09 s
9	352	0.37 s
10	724	1.6 s
11	2,680	7.5 s
12	14,200	36 s

Counting means exploring the whole tree, so heuristics help less: each extra row multiplies the time by about four or five.

Exporting a solution to JSON

```
write_json(File, N, Cols) :-
    Dict = _{n: N, solution: Cols},
    setup_call_cleanup(
        open(File, write, S, [encoding(utf8)]),
        json_write_dict(S, Dict, [width(0)]),
        close(S)).
```

```
{"n":8, "solution": [1, 5, 8, 6, 3, 7, 2, 4 ]}
```

library(http/json) turns a dict into JSON. The file stores the board size and one column per row, counted from 1.

The Python version writes the same format, so one viewer reads both.

Running it

```
$ swipl -q nqueens.pl 8
 Q . . . . . . .
 . . . . Q . . .
 . . . . . . . Q
 . . . . . Q . .
 . . Q . . . . .
 . . . . . . Q .
 . Q . . . . . .
 . . . Q . . . .
Wrote one solution for N = 8 to queens8.json
```

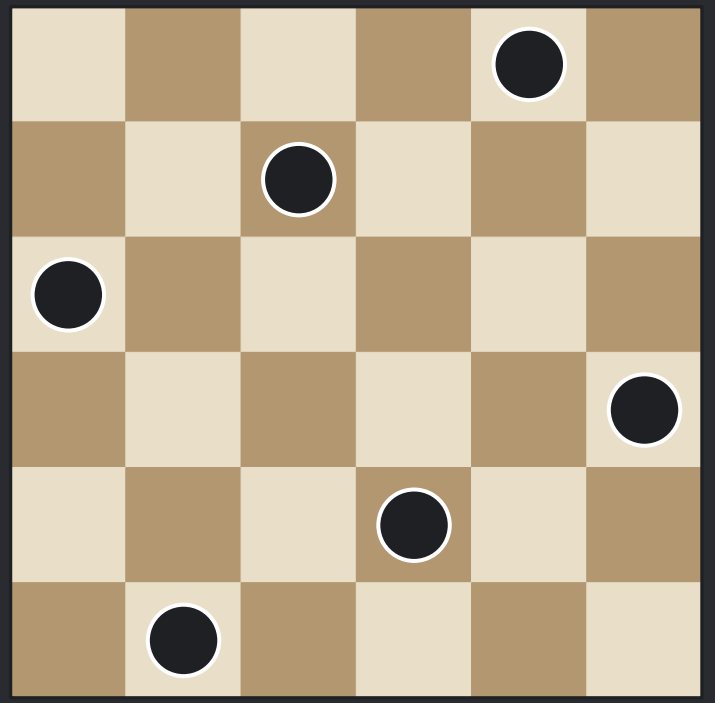
With N: solve, print, write queensN.json.

Without N, `swipl nqueens.pl` opens the interactive toplevel with everything loaded:

```
?- queens(8, Cols).
?- count_solutions(8, K).
?- time_first(30, ff).
```

Python with python-constraint

A pure-Python CSP library: pip install python-constraint, works in Colab



The library in four calls

Call	What it does
`Problem(solver)`	a new CSP, with the solver that will search it
`addVariables(vars, domain)`	variables that share one domain
`addConstraint(c, vars)`	a constraint object, or any function returning True / False
`getSolution()`	one solution as a dict, or None
`getSolutions()`	a list with every solution

```
from constraint import Problem, AllDifferentConstraint, BacktrackingSolver, MinConflictsSolver
```

The model

```
def build(n, solver=None):
    """The CSP for n queens, without searching."""
    problem = Problem(solver or BacktrackingSolver())
    rows = range(n)
    problem.addVariables(rows, range(n))           # each row gets a column
    problem.addConstraint(AllDifferentConstraint(), rows)  # different columns
    for i in rows:                                 # different diagonals
        for j in range(i + 1, n):
            problem.addConstraint(
                lambda qi, qj, d=j - i: abs(qi - qj) != d, (i, j))
    return problem
```

Same variables and domains as in Prolog, counted from 0. Columns: one `AllDifferentConstraint`. Diagonals: one small function per pair of rows.

No sums and differences: pairwise diagonals

python-constraint can only apply AllDifferent to variables, not to expressions like Qi + i. So the diagonal rule is checked pair by pair.

$$N(N-1)/2$$

functions for the diagonals, one per pair of rows

N	pairwise constraints	in Prolog
8	28	3 all_different
20	190	3 all_different
100	4,950	3 all_different

A Python trap: the d = j − i default

```
# correct: d is fixed when the lambda is created
lambda qi, qj, d=j - i: abs(qi - qj) != d

# wrong: j and i are read later, after the loops end
lambda qi, qj: abs(qi - qj) != j - i
```

40,320

“solutions” for N = 8 with the wrong version: every permutation

Python closures look up i and j when they run, not when they are created. By then the loops have finished, j − i is 0, and every diagonal constraint is true.

Running it

```
$ python3 nqueens.py 8
. . . . . . . Q
. . . Q . . . .
Q . . . . . . .
. . Q . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . . . Q .
. . . . Q . . .
Wrote one solution for N = 8 to queens8.json

$ python3 nqueens.py 8 --count
N = 8: 92 solutions
```

A different first solution than Prolog's: this solver tries values from the last column backwards.

`solve(n, forwardcheck=True)` builds the problem with `BacktrackingSolver(forwardcheck)` and returns the columns as a list.

Inside BacktrackingSolver

```
# Mix the Degree and Minimum Remaing Values (MRV) heuristics
lst = [
    (-len(vconstraints[variable]), len(domains[variable]), variable)
    for variable in domains
]
lst.sort()
```

From the library's own source code.

Next variable: most constraints first (degree), then fewest values left (MRV).

forwardcheck=True, the default, removes values from the neighbours' domains after each assignment.

In N queens every row has the same number of constraints, so in practice it is MRV with forward checking.

Forward checking, on and off

N	forwardcheck=True	forwardcheck=False
8	0.001 s	0.002 s
16	0.002 s	0.31 s
20	0.004 s	7.6 s
22	0.001 s	73 s
50	0.074 s	> 20 s
100	0.11 s	> 20 s

Time to the first solution.

Without forward checking the domains never shrink, so MRV has nothing to compare: every row looks equally good, and the search falls back to plain order.

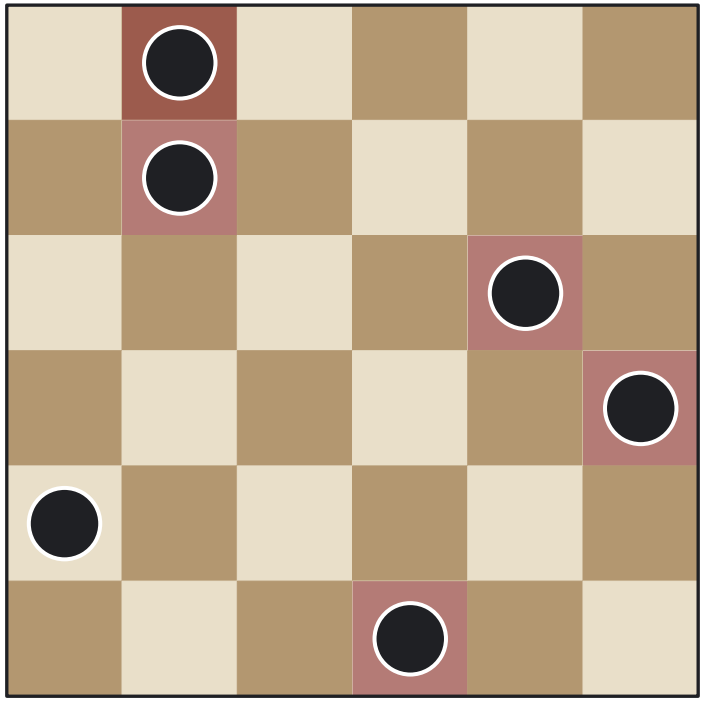
Same answers in both languages

N	solutions	Prolog, leftmost	Python, MRV + FC
8	92	0.09 s	0.02 s
10	724	1.6 s	0.32 s
11	2,680	7.5 s	0.88 s
12	14,200	36 s	4.4 s

Not just the same counts: for N = 6, 8 and 10 we compared the solution lists one by one, and they are identical.

The time gap comes from the search options used for counting, not from the language.

Local search: min-conflicts



Marked: queens in conflict.

Start with every queen placed, conflicts allowed.

Repeat: pick a queen in conflict and move it, within its row, to the column with the fewest conflicts.

Stop when there are no conflicts, or after a fixed number of steps.

```
Problem(MinConflictsSolver(steps=1000))
```

Local search needs the right model

N	with AllDifferentConstraint	pairwise only
8	49 / 50	28 / 30
20	0 / 20	20 / 20
50	0 / 10	10 / 10

Runs that found a solution within 1,000 steps.

nqueens.py keeps both models: `build()` for backtracking and `build_pairs()` for min-conflicts.

AllDifferentConstraint counts as one violated constraint whether two queens share a column or ten do. Min-conflicts cannot tell which queen is worse, so it wanders. One constraint per pair gives it a real conflict count.

Where backtracking stalls, local search keeps going

N	backtracking (MRV + FC)	min-conflicts
100	0.11 s	0.30 s
150	> 20 s	—
200	> 20 s	2.2 s
500	> 20 s	25 s

Median time for min-conflicts with the pairwise model; every run solved.

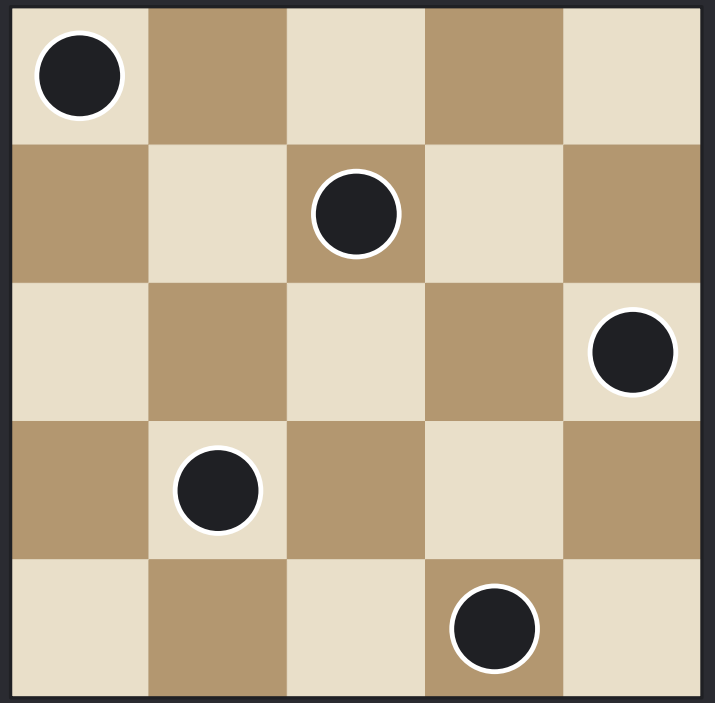
The price:
it can give up without an answer (2 of 30 runs for N = 8),
it cannot prove that no solution exists,
and it cannot count solutions.

The two implementations side by side

	Prolog, clp(fd)	**Python, python-constraint**
Diagonals	3 global all_different	N(N−1)/2 pairwise functions
Search	labeling/2 with options	BacktrackingSolver
Heuristics	you choose: leftmost, ff, ffc	built in: degree, then MRV
Propagation	always on	forwardcheck=True / False
Local search	—	MinConflictsSolver
First solution, N = 100	0.46 s with ff	0.11 s

The lab

Four experiments. For each one, write your prediction before you run anything



Setup

```
# Prolog: SWI-Prolog, nothing else to install
swipl nqueens.pl

# Python: one library
pip install python-constraint
python3 nqueens.py 8
```

github.com/ferdiex/csp — folder nreinas

The Python file also runs in Colab: upload it, then ! pip install python-constraint.

File	Contents
`prolog/nqueens.pl`	model, search, counting, timing, JSON
`python/nqueens.py`	same, plus forward checking switch and min-conflicts
`ver_tablero.html`	opens any queensN.json and checks it

Experiment 1: how many solutions?

N	4	5	6	7	8	9	10
your prediction							
count_solutions							

Run it in both languages and check that they agree.

Why does N = 6 have fewer solutions than N = 5? Look at the 4 solutions of N = 6 in the viewer.

How much longer does each extra row take to count?

Experiment 2: does first fail matter?

```
?- time_first(20, leftmost).
?- time_first(20, ff).
?- time_first(20, ffc).
```

Stop a search that takes too long with Ctrl + C, then a.

Try N = 8, 12, 16, 20, 21, 22, 30 and 100 with each option.

Does the time always grow with N? Find one N that is faster than a smaller one.

Relate ff to MRV on the map page from last class.

Experiments 3 and 4: forward checking and local search

```
import nqueens as nq
nq.time_first(20, forwardcheck=True)
nq.time_first(20, forwardcheck=False)
```

```
$ python3 nqueens.py 100 --min-conflicts
$ python3 nqueens.py 200 --min-conflicts
```

3. Compare the two times for N = 8, 16, 20 and 22. Why does MRV stop helping when forward checking is off?

4. Compare backtracking and min-conflicts for N = 50, 100 and 200. Then try min-conflicts on N = 3: what does it answer, and what does that tell you?

What to hand in

Item	Details
Predictions	written before running, for each experiment
Tables	the measured values, with your machine and N used
Two boards	queens8.json from each language, opened in the viewer
Answers	the questions of experiments 1 to 4, in a few lines each
One paragraph	when would you choose clp(fd), python-constraint or min-conflicts?

Times change from machine to machine; what should hold everywhere are the comparisons.

State the problem; let the solver search

The model decides what the search can see. The heuristics and the propagation decide how fast it finds it.

